

Assembly

3CIIN

Maggio 2022

Fonti: Appunti Prof.Salvi,
Prof.ssa Secchi

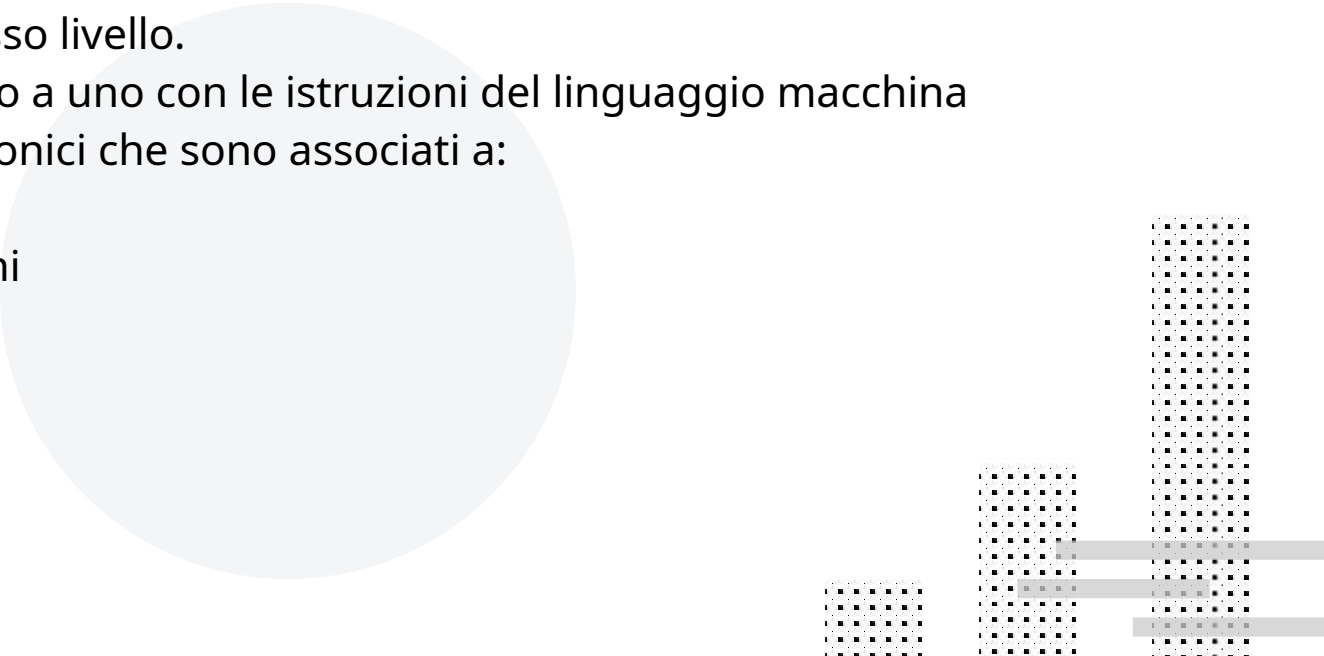


Caratteristiche del linguaggio Assembly

E' un linguaggio di basso livello.

C'è corrispondenza uno a uno con le istruzioni del linguaggio macchina

Utilizza simboli mnemonici che sono associati a:

- istruzioni
 - sequenze di istruzioni
 - indirizzi di memoria
 - aree di memoria
 - dispositivi di I/O
- 

Processore HC08

HC08 è un **Microcontrollore** (un sistema integrato) composto da:

- CPU a 8 bit
- ROM
- RAM 128 byte
- I/O
- Periferiche

Ha 64Kb di memoria disponibili (256 pagine di 256 celle)

I primi **64 bytes**, da 0x00 (0) a 0x3f (63) sono per **registri I/O**

128 bytes sono per la **RAM** tra la locazione 0x80 (128) e la 0xff (255)

4096 bytes per la **ROM programmabile** (Flash Eprom) da locazione 0xEE00 a 0xFDFF, 416 bytes per la ROM non riscrivibile tra le locazioni 0xFE10 e 0xFFAF

I registri

Registro: è una variabile (può contenere un solo dato alla volta)

- Accumulator **A** memorizza valori e risultati di operazioni aritmetico-logiche
A → 8 bit
- Index **H:X** contiene indirizzi di memoria o indici di array
X → 8 bit Individua le prime 256 celle di memoria (0..255) e consente indirizzamento veloce (a pagina zero)
H:X → 16 bit Indirizza tutti i 64Kb di memoria
- Program Counter **PC** contiene indirizzo della prossima istruzione del programma
16 bit
- Stack Pointer **SP** contiene l'indirizzo della locazione di memoria occupata dal top dello stack
16 bit
- Conditioner Code **CCR** Contiene una serie di bit relativi al risultato dell'ultima operazione aritmetico/logica eseguita (come il riporto, la parità, il segno ecc.) o che influenzano il comportamento della CPU
8 bit

Istruzioni

Vengono tradotte dall'Assemblatore in istruzioni macchina. Ogni istruzione è composta in generale da:

- una Etichetta (Label)
- un Codice Operativo (Operation Code)
- uno o più operandi (Operands)

Esempio:

Label	Op.Code	Operands
START:	MOV	AX

Etichette (label)

Sono identificatori associati ad una istruzione; l'assemblatore le sostituisce con l'indirizzo dell'istruzione che rappresentano.

- permettono di trovare più facilmente un punto del programma
- facilitano la modifica del programma

Esempio:

Label
START:

Op.Code
MOV

Operands
#10, num1

Codici operativi (Op.code)

E' la stringa mnemonica dell'istruzione Assembly, specifica l'operazione che deve essere eseguita dalla CPU

Non può mai mancare in una riga di programma.

Esempio:

Label
START:

Op.Code
MOV

Operands
#10, num1

Operandi (Operands)

Possono non esserci.

Esempio:

Label
START:

Op.Code
MOV

Operands
#10, num1

Creazione di un programma Assembly

Il processo di creazione di un programma Assembly passa attraverso le seguenti fasi:

- scrittura del programma sorgente tramite un normale **EDITOR** in un file con estensione .asm
- assemblaggio, tramite un **ASSEMBLATORE** del file sorgente e generazione di un file oggetto con estensione .OBJ
- creazione del file eseguibile con estensione .EXE, tramite un **LINKER**

Struttura un programma Assembly

```
;ese.asm
; Descrizione del programma
; Costanti
STARTPGM    =    0xee00    ; L'inizio della ROM
STARTRAM    =    0x80     ; L'inizio della RAM
RESETVECT   =    0xffff   ; La posizione del vettore di reset
;L'area per le variabili
                .area    DATA (ABS)
                .org     STARTRAM
; Le definizioni delle variabili
; L'area del programma
                .area    PROGRAMMA (ABS)
                .org     STARTPGM
; Le istruzioni del programma
MAIN:        ; la "funzione" main
; il vettore di reset
                .area    RESET (ABS)
                .org     RESETVECT
                .word    MAIN    ; Puntatore a MAIN per il reset
```

Direttive
all'assemblatore

Programma assembly che esegue la somma CC di due valori AA e BB.

```
; i commenti sono preceduti da ";" non vengono considerati  
; ese.asm  
; somma AA + BB ed il risultato viene messo in CC
```

```
; COSTANTI che definiscono indirizzi di memoria da cui partono:
```

```
STARTPGM    = 0xee00          ; ipotetica ROM (flash)  
RESETVECT   = 0xffff         ; vettore di reset (punt a prima riga da eseguire)  
STARTDATI   = 0xb1000000      ; Area dati (primo indirizzo di RAM  
                               ; disponibile) in esad = 80 in dec = 128
```

```
; definisco area che chiamo DATI  
; per raggruppare i dati (VARIABILI e Procedure)
```

```
    .area DATI (ABS)  
    .org STARTDATI          ; indirizzo da cui parte area di definizione DATI  
  
AA:      .blkb      1      ; definisco tre variabili AA BB CC (alloco spazio)  
BB:      .blkb      1      ; blkb definisce blocco di n bytes non iniz.  
CC:      .blkb      1      ; AA BB CC occupano ciascuno 1 bytes
```

; definisco area che chiamo PROGRAMMA per raggruppare le **ISTRUZIONI** operative

```
.area    PROGRAMMA (ABS)
.org     STARTPGM          ; inizio area PROGRAMMA

MAIN: LDA #0b10000001      ;LoaDA mette in A (registro accumulatore) il valore 129
    STA AA                ;SToreA mette in AA il valore che trova
                           ;nell'accumulatore A --> AA=129

    LDA #1                 ;LoaDA mette in A (registro accumulatore) il valore 1
    STA BB                ;SToreA mette in BB il valore che trova
                           ;nell'accumulatore A --> BB=1

    LDA AA                ;LoaDA mette in A (registro accum.) il valore AA (129)
    ADD BB                ;aggiunge il valore BB (1) in A (registro accumulatore)
                           ;che avrà quindi valore 130
    STA CC                ;SToreA mette in CC il valore che trova
                           ;nell'accumulatore A --> CC=130 (CC=AA+BB)

FINE: BRA FINE             ;Loop infinito per fermare il programma
```

; definisco area che chiamo RESET per raggruppare le istruzioni di reset

```
.area    RESET (ABS)
.org     RESETVECT
.word    MAIN              ; alloca 2 byte con valore MAIN (locazione iniz. del prg)
```

Definizione costanti*

NELEM = 10 costante decimale

MASK = 0b01010101 costante binaria

STARTDATI = 0x0080 costante esadecimale

*Inserire il codice prima di tutte le aree definite.

Definizione variabili e procedure* (allocazione celle della RAM)

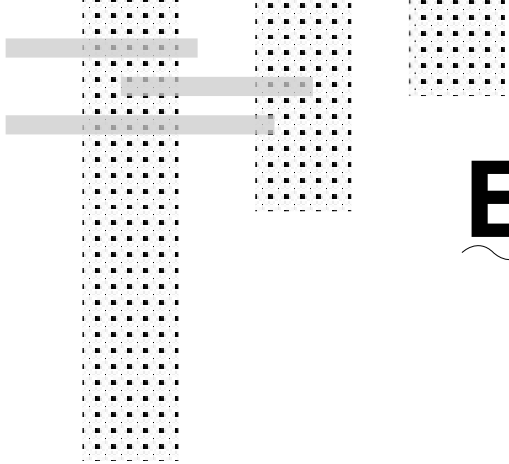
MAX: .blkb 1

variabile con un blocco di 1 byte (non inizializzata)

VETT: .blkb 8

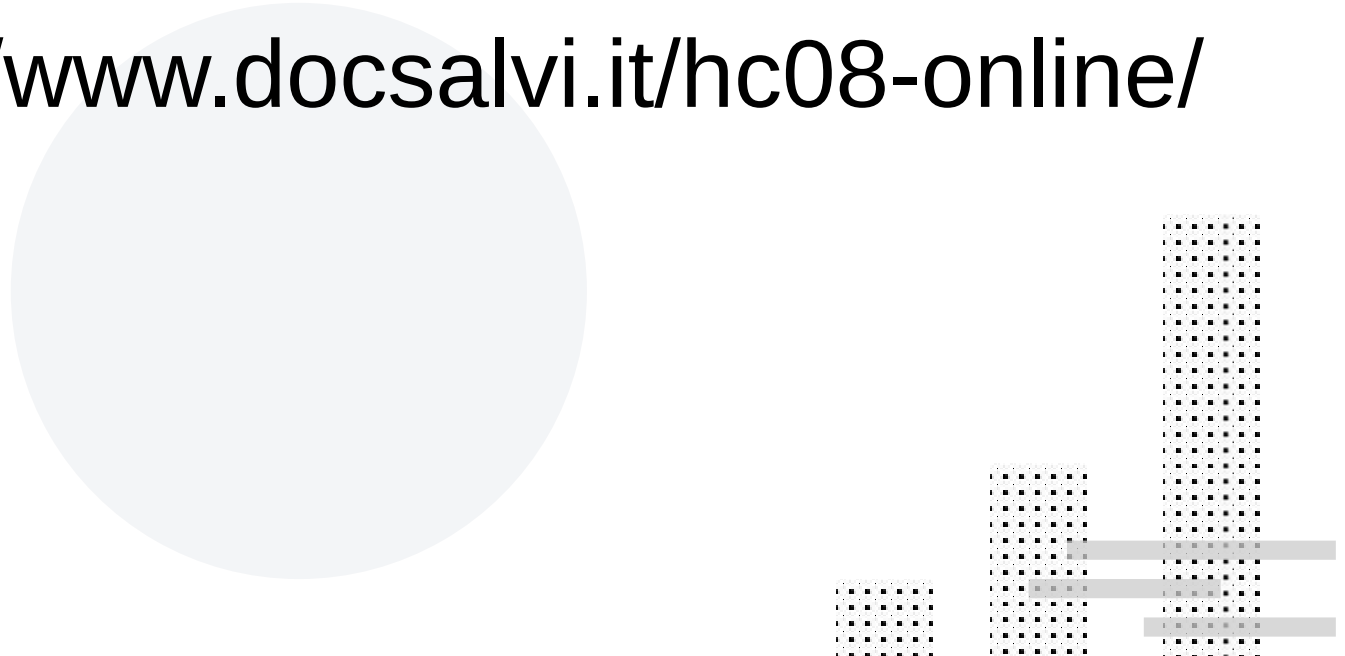
un vettore con 8 blocchi (celle) da 1 byte ciascuno

*Inserire il codice nell'area dati (RAM) riservata



Emulatore HC08

<https://www.docsalvi.it/hc08-online/>



Codice

ee00	ae	81	LDX #81
ee02	f6		LDA ,X
ee03	b7	80	STA \$80
ee05	5c		INCX
ee06	f6		LDA ,X
ee07	b1	80	CMP \$80
ee09	93	02	BLE ee0d
ee0b	b7	80	STA \$80
ee0d	5c		INCX
ee0e	51	89 02	CBEQX #89,ee13
ee11	20	f3	BRA ee06
ee13	20	fe	BRA ee13
ee15	ff		STX ,X
ee16	ff		STX ,X
ee17	ff		STX ,X
ee18	ff		STX ,X

Comandi

s - step
r - reset
t - Commuta tasto
f - Ridisegna display
q - esci

Registri

Accumulatore..... : ff (255)
H:X..... : 0000
Stack Pointer..... : 00ff
Program Counter..... : ee00
Condition Code Register : vllhInxc

Dati

0080	ff	15	08	0c	1b	6c	01	60	l.`
0088	02	ff	ff	ff	ff	ff	ff	ff	
0090	ff	ff	ff	ff	ff	ff	ff	ff	
0098	ff	ff	ff	ff	ff	ff	ff	ff	
00a0	ff	ff	ff	ff	ff	ff	ff	ff	
00a8	ff	ff	ff	ff	ff	ff	ff	ff	
00b0	ff	ff	ff	ff	ff	ff	ff	ff	
00b8	ff	ff	ff	ff	ff	ff	ff	ff	
00c0	ff	ff	ff	ff	ff	ff	ff	ff	
00c8	ff	ff	ff	ff	ff	ff	ff	ff	
00d0	ff	ff	ff	ff	ff	ff	ff	ff	

I/O

s 0 0
12345678901234567890

Tasto: ☐

Nel riquadro intitolato [**Codice**] in rosso c'è la prossima istruzione da eseguire (quella il cui indirizzo è in Program Counter). La prima colonna indica l'indirizzo di memoria in esadecimale.

Codice

```

ee00 ae 81      LDX #81
ee02 f6        LDA ,X
ee03 b7 80     STA $80
ee05 5c        INCX
ee06 f6        LDA ,X
ee07 b1 80     CMP $80
ee09 93 02     BLE ee0d
ee0b b7 80     STA $80
ee0d 5c        INCX
ee0e 51 89 02  CBEQX #89,ee13
ee11 20 f3     BRA ee06
ee13 20 fe     BRA ee13
ee15 ff        STX ,X
ee16 ff        STX ,X
ee17 ff        STX ,X
ee18 ff        STX ,X

```

Comandi

```

s - step
r - reset
t - Commuta tasto
f - Ridisegna display
q - esci

```

Registri

```

Accumulatore..... : ff (255)
H:X.....           : 0000
Stack Pointer.....   : 00ff
Program Counter..... : ee00
Condition Code Register : v11hInxc

```

Dati

0080	ff	15	08	0c	1b	6c	01	60	1..`
0088	02	ff	ff	ff	ff	ff	ff	ff	
0090	ff	ff	ff	ff	ff	ff	ff	ff	
0098	ff	ff	ff	ff	ff	ff	ff	ff	
00a0	ff	ff	ff	ff	ff	ff	ff	ff	
00a8	ff	ff	ff	ff	ff	ff	ff	ff	
00b0	ff	ff	ff	ff	ff	ff	ff	ff	
00b8	ff	ff	ff	ff	ff	ff	ff	ff	
00c0	ff	ff	ff	ff	ff	ff	ff	ff	
00c8	ff	ff	ff	ff	ff	ff	ff	ff	
00d0	ff	ff	ff	ff	ff	ff	ff	ff	

I/O

s

0

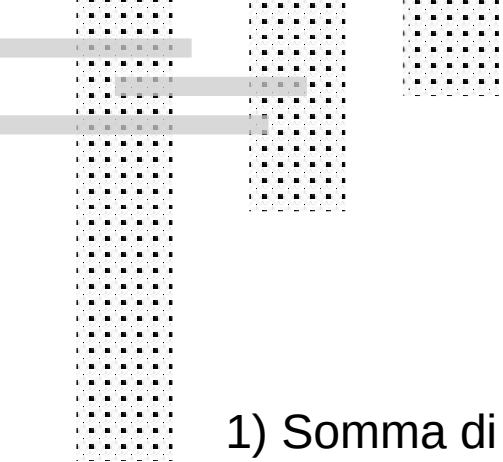
0

12345678901234567890

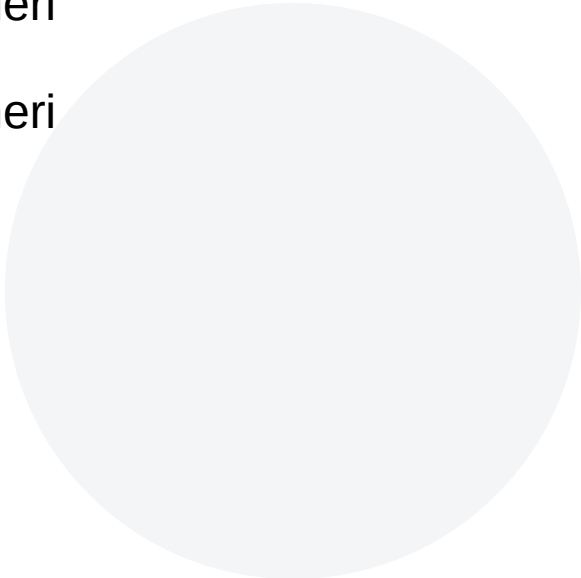
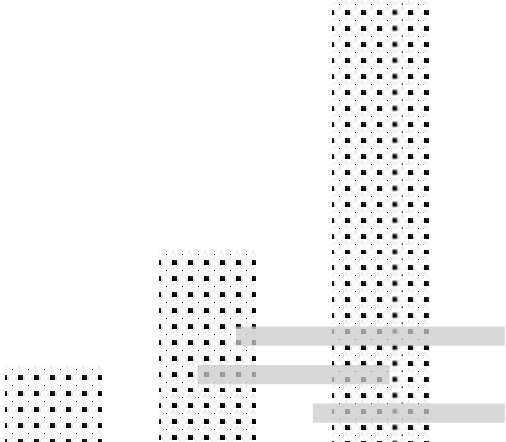
Tasto: ☐

Nel riquadro [**Registri**] sono elencati tutti i registri della CPU HC08. Il valore dell'accumulatore è visualizzato sia in esadecimale che in decimale.

Nel riquadro intitolato [**Dati**] La prima colonna indica l'indirizzo della memoria dati in esadecimale. Dalla seconda colonna si vede il contenuto delle variabili utilizzate dal programma.



Esercizi 10/5

- 1) Somma di 3 numeri
 - 2) Massimo di 2 numeri
 - 4) Massimo di 3 numeri
- 
- 

Esercizi 18/5

- 1) Confrontare due numeri num1 e num2 . Se $\text{num1} > \text{num2}$ in num3 mettere la loro somma altrimenti la loro differenza
- 2) Calcolare la somma dei primi 15 numeri
- 3) Calcolare il prodotto di 2 numeri tramite somme successive e mettere il risultato in una variabile
- 4) Calcolare la somma dei primi 10 numeri dispari e la differenza di quelli pari

IF

<pre><u>If</u> (A<M){ a } b c</pre>	<pre>SE: <u>CMP</u> M <u>BGE</u> MORE a MORE: b c</pre>	Compara A con M (A-M) e se $A \geq M$, salta a label MORE ed esegue b e c, altrimenti esegue istruzioni a e poi b c La condizione va <u>invertita</u> rispetto al C
<pre><u>If</u> (A>M){ a }else{ b } c</pre>	<pre>SE: <u>CMP</u> M <u>BLE</u> ELSE a BRA <u>END_IF</u> ELSE: b <u>END_IF</u>: c</pre>	Compara A con M (A-M) e se $A \leq M$, salta a label ELSE ed esegue b e c, altrimenti esegue istruzioni a e c La condizione va <u>invertita</u> rispetto al C

WHILE

```
While (A==M) {
```

```
    istruz
```

```
}
```

```
LOOP: CMP M
```

```
    BNE(**) FINE
```

```
    istruz
```

```
    BRA LOOP
```

```
FINE:
```

```
** BEQ
```

```
** BNE
```

```
** BLE
```

```
** BLT
```

```
** BGE
```

```
** BGT
```

Compara (CoMPare) valore del registro A con valore di M

se A!=M Branch (salta) a FINE

altrimenti (A==M) ed esegue istruz

torna a LOOP

A=M Equal

A!=M NotEqual

A<=M LessOrEqual

A<M LessThan

A>=M GreaterOrEqual

A>M GreaterThan

DO-WHILE

```
Do{  
    istruz  
} while (A==M);
```

```
LOOP:  
    istruzione  
  
    CMP M  
    BEQ LOOP
```

Esegue istruzioni,

compara (CoMPare) A con M e
se A==M torna su a label LOOP
altrimenti prosegue

FOR

```
for (I=1; I<N; I++){  
    istruz  
}
```

```
MOV #1, I  
LDA I  
FOR:  
  
CMP #N, FINEFOR  
  
BGE FINEFOR  
    istruz  
    INC I  
    LDA I  
    BRA FOR  
FINEFOR:
```

Memorizza il valore 1 nella variabile I e carica in A il valore di I; I=1

Compara A con il valore cost N (confronta I con N)
Se I>=N esce dal ciclo (va a FINEFOR - il contrario rispetto al C), altrimenti se I<N esegue istruz, incrementa I e ritorna al controllo (label FOR)

Costanti e variabili

Costanti:

#128 indica il valore 128 in decimale
#0b10000000 indica il valore 10000000 in binario (128 in dec)

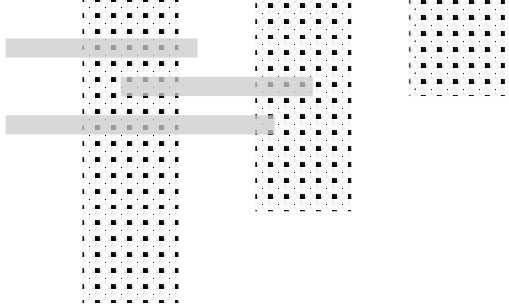
Attenzione, un valore costante senza #, indica indirizzo, perciò:
81 indica indirizzo di memoria 81

COST indica indirizzo della costante COST
#COST indica valore contenuto nella costante COST

Variabili:

VAR indica il valore contenuto nella cella di memoria identificata dal nome della variabile VAR

#VAR indica l'indirizzo di cella della variabile VAR
*VAR come sopra ma indirizzamento a 8 bit (anziché 16) pagina zero



Registri

Registri:

X indica indirizzo memorizzato nel registro Indice
A indica il valore memorizzato nel registro Accumulatore

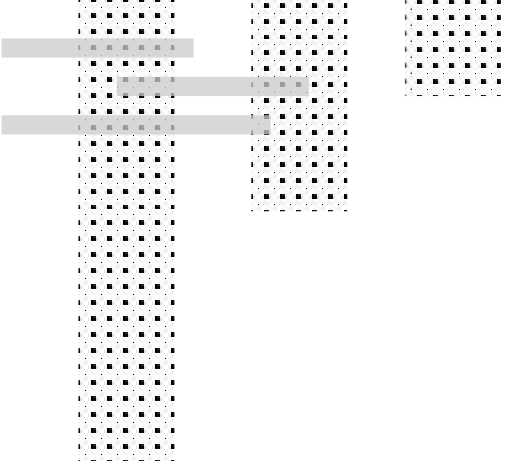
Array:

#VETT indica l'indirizzo della prima cella del vettore VETT
VETT ,X indica il valore contenuto del vettore all'indice X
 ,X indica valore dell'elemento puntato da X

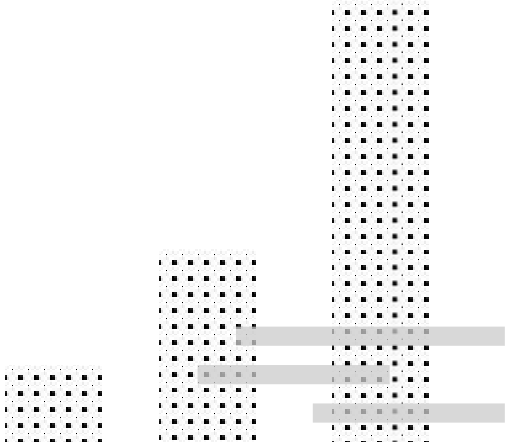


Esercizi di ripasso

- 1) Definire 3 variabili (num1, num2 e num3) e scriverle nelle tre variabili (primo, secondo e terzo) in ordine crescente.
- 2) Calcolare media dei primi 10 numeri



I vettori



Indirizzamento immediato

Le seguenti istruzioni usano l'indirizzamento immediato:

LDA #10 l'accumulatore sarà caricato con il valore costante che segue (immediatamente) l'istruzione, cioè 10.

LDX #VAR il registro indice sarà caricato con **l'indirizzo di memoria** identificato dal nome di variabile VAR (perché i nomi di variabili identificano indirizzi di celle di memoria)

LDA #COST l'accumulatore sarà caricato col **valore contenuto nella costante** COST (alla quale abbiamo associato un valore nella sezione apposita del programma)

Indirizzamento diretto

In questo caso si recupera il valore contenuto in una cella di memoria utilizzando l'indirizzo specificato.

Le seguenti istruzioni usano l'indirizzamento **diretto**:

LDA VAR l'accumulatore sarà caricato (direttamente) con il **valore** preso dall'indirizzo di memoria identificato dal nome della variabile VAR

LDX COST il registro indice sarà caricato con l'**indirizzo** di memoria identificato dalla costante COST)

Costanti e variabili

Costanti:

#128

indica il **valore** 128 in decimale

#0b10000000

indica il **valore** 10000000 in binario (128 in dec)

ATTENZIONE: un valore costante senza #, indica indirizzo, perciò: 81 indica indirizzo di memoria 81

COST

indica l'**indirizzo** della costante COST

#COST

indica il **valore** contenuto nella costante COST

Variabili:

VAR indica il **valore** contenuto nella cella di memoria identificata dal nome della variabile VAR

#VAR

indica l'**indirizzo** di cella della variabile VAR

*VAR

come # (indirizzo) ma indirizzamento a 8 bit (anziché 16) a pagina

zero

I vettori - allocazione e inizializzazione

in area dati (RAM) o in area programma (ROM)

VETT: .blkb 8 ;un vettore di otto celle da 8 bit
ENDVETT: ;posizione dopo la fine del vettore

VETT: .byte 0b0000 ;vettore di 3 celle
 .byte 0b1111 ;con dati costanti scritti in binario
 .byte 0b0000
ENDVETT:

VETT: .byte 0x01 ;vettore di 3 celle
 .byte 0x4F ;con dati costanti scritti in esadecimale
 .byte 0x80
ENDVETT:

I vettori - lettura

Lettura di un vettore di 6 elementi con **metodo indice associato**

```
LDX #0  
LOOP: CBEQX #N,FINE
```

```
LDA VETT ,X
```

```
...  
INCX  
BRA    LOOP
```

```
FINE:
```

pongo indice $X = 0$

confronta registro X con valore della costante N
se vero vado a FINE

indirizzamento con indice, metto il valore
dell'elemento X di VETT alla posizione X, in A

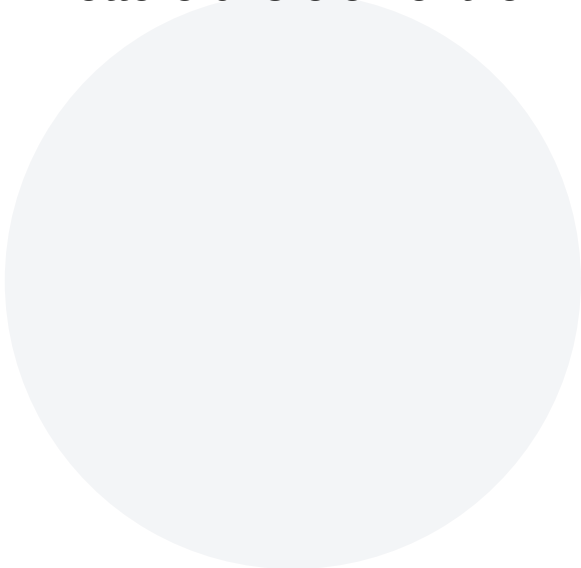
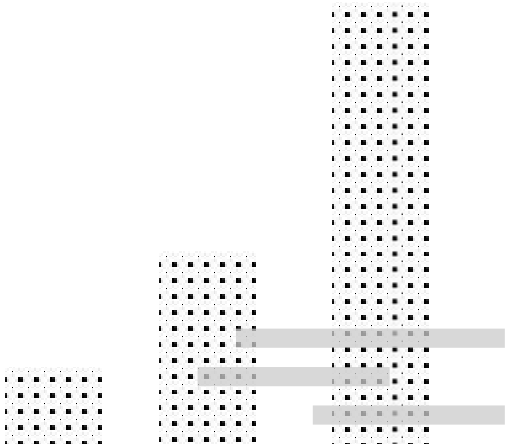
I vettori - lettura

Lettura di un vettore con metodo puntatore a cella

	LDX #VETT	indirizzo del primo elemento del vettore VETT
LOOP:	CBEQX #ENDVETT,FINE	confronta X con indirizzo di ENDVETT, se vero vado a FINE
	LDA ,X	indirizzamento diretto come puntatore, metto il valore che sta nella cella puntata da X in A
	...	
	INCX	incremento di una cella (1 byte) il valore di X
BRA	LOOP	
FINE:		



Esercizi lettura

- 1) Leggi i valori di un vettore di 5 elementi e calcola la media
 - 2) Leggi i valori di un vettore di 8 elementi e metti in una variabile il maggiore
- 
- 

I vettori - scrittura

Scrittura di un vettore con metodo indice associato

	LDX #0	Inizializzo registro indice X al valore 0
	LDA #10	Metto in A il valore di partenza
LOOP:	STA VETT,X	Metto valore di A nella posizione X di VETT
	INCA	Incremento il valore di A
	INCX	Incremento il valore di X
	CBEQX #N, END	Se l'indice è arrivato a N, esco
	BRA LOOP	Altrimenti rieseguo

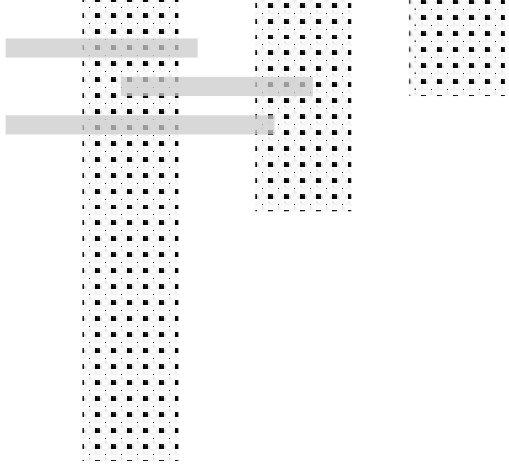
Carica un vettore di 10 elementi con 10, 11, 12, ecc..

I vettori - scrittura

Scrittura di un vettore con metodo puntatore a cella

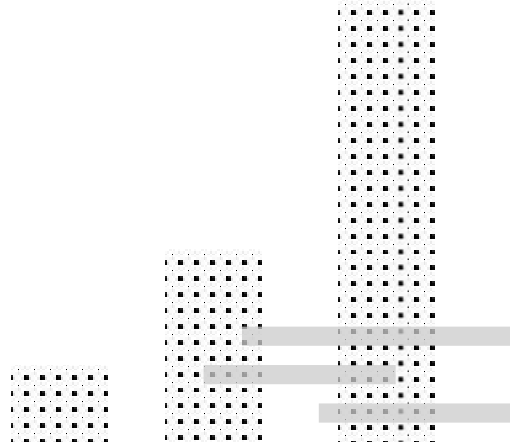
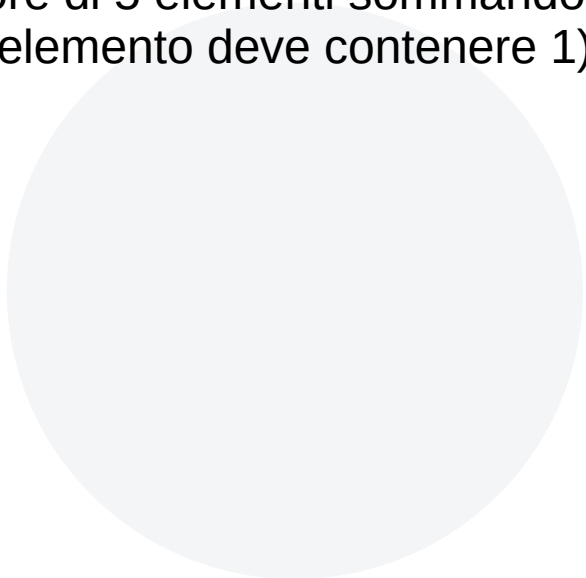
	LDX #VETT	indirizzo del primo elemento del vettore VETT
	LDA #10	Metto in A il valore di partenza
LOOP:	STA ,X	Metto valore di A nella cella a indirizzo X
	INCA	Incremento il valore di A
	INCX	Incremento il valore di X
	CBEQX #ENDVETT, END	Se l'indice è arrivato a fine VETT, esco
	BRA LOOP	Altrimenti torna a LOOP

Carica un vettore di N (costante) elementi con 10, 11, 12, ecc..



Esercizio scrittura

1) Inizializza un vettore di 5 elementi sommando 3 al valore nella cella precedente (il primo elemento deve contenere 1). Quindi il vettore conterrà 1,4,7,10,13





Esercizi lettura/scrittura

- 1) Inizializza un vettore di 10 elementi con i primi 10 numeri pari e poi calcola la somma con un successivo ciclo
 - 2) Inizializza un vettore di 6 elementi partendo da 1. Gli elementi successivi sono la somma dei due precedenti (Fibonacci). Quindi il vettore conterrà 1,1,2,3,5,8
- 